飞腾嵌入式 E2000/D2000 PCIe EP 模式使用手册

（V1.1）

Phytium 飞腾

飞腾信息技术有限公司
www.phytium.com.cn
2025 年 3 月 20 日

版权所有 © 飞腾信息技术有限公司 2025。 保留一切权利。

未经本公司同意，任何单位、公司或个人不得擅自复制、翻译、摘抄本文档内容的部分或全部，不得以任何方式或途径进行传播和宣传。

商标声明

Phytium 和其他飞腾商标均为飞腾信息技术有限公司的商标。

本文档提及的其他所有商标或注册商标，由各自的所有人拥有。

特别提示

本文档仅作为使用指导，飞腾对本文档内容不做任何明示或暗示的声明或保证。本文档中的所有陈述、信息和建议不构成任何明示或暗示的担保。

由于产品版本升级或其他原因，本文档内容会不定期进行更新，如有变更，恕不另行通知。

最新技术资源

飞腾嵌入式软件开源社区 https://gitee.com/phytium_embedded

或者访问飞腾软件开发者平台 https://service.phytium.com.cn/developer/

Phytium飞腾

目录

1 概述 1
2 EP 模式内核编译 1
2.1 环境准备 1
2.1.1 准备 Linux kernel 源码 1
2.1.2 Config 选项配置 1
2.1.3 设备树修改 2
2.1.4 编译内核 3
2.1.5 固件支持 3
2.1.6 编译 pcitest 工具 3
2.1.7 硬件环境 3
3 功能验证 4
3.1 EP 端 4
3.1.1 EP 端驱动加载 4
3.1.2 创建 pci-epf-test 设备 5
3.2 RC 端 6
3.2.1 RC 端驱动加载 6
3.2.2 执行测试程序 6
4 注意事项 8
5 常见问题 8
A 术语 9
B 更新记录 9

Phytium飞腾

1 概述

在 PCIe 架构中，有两种关键的工作模式：RC（Root Complex）和 EP（Endpoint），分别对应主模式和从模式。RC 模式下的设备用于连接 CPU/内存子系统和 I/O 设备，负责管理和控制与 PCIe 总线相连的所有设备。EP 模式下的设备代表 PCIe 架构中的终端设备，负责响应来自 RC 或其他设备的数据请求。

飞腾 E2000 和 D2000 默认为 RC 模式，本文档描述如何编译使用 EP 模式的 Linux 内核，以及在 Linux 环境下测试 EP 模式的基本功能。

2 EP 模式内核编译

2.1 环境准备

2.1.1 准备 Linux kernel 源码

代码发布地址：

https://gitee.com/phytium_embedded/phytium-linux-kernel

```
git clone https://gitee.com/phytium_embedded/phytium-linux-kernel.git

确保包含以下 commit
linux-5.10  分支：  90dc7442be674a25a839637056daeec6372527af
linux-4.19  分支：  f5d1272ed29c2b6362cb9b09c84bb2d4b0c03868
linux-6.6   分支：  保证在 tag v3.1 或之后

内核编译可以参考如下链接：
https://gitee.com/phytium_embedded/phytium-linux-
kernel/wikis/%E8%B0%83%E8%AF%95%E4%B8%8E%E8%B0%83%E4%BC%98/%
E5%8D%95%E7%8B%AC%E7%BC%96%E8%AF%91Linux%20Kernel
```

2.1.2 Config 选项配置

在 ARM64 或 X86 主机上安装和设定好编译器和其他环境变量后，首先加载默认的飞腾 config

```
make phytium_defconfig
```

然后再修改

```
make menuconfig
```

EP 端：将 PCI 控制器配置为 EP 功能，必须启用以下配置选项。

```
CONFIG_PCIE_PHYTIUM_EP=y
CONFIG_PCI_ENDPOINT=y
CONFIG_PCI_ENDPOINT_CONFIGFS=y
CONFIG_PCI_EPF_TEST=m
```

RC 端：为了使用“Endpoint Test”PCI 设备，必须启用以下配置选项。

```
CONFIG_PCI_ENDPOINT_TEST=m
```

2.1.3 设备树修改

EP 端：在设备树中设置 EP 信息并使能：

```
E2000 设备树（arch/arm64/boot/dts/phytium/e2000q-demo-board.dts）：
        ep0: ep@31040000 {
                compatible = "phytium,pcie-ep-2.0";
                reg = <0x0 0x31040000 0x0 0x10000>,          //使用的控制器为 PEU.C0
                        <0x11 0x00000000 0x1 0x00000000>,
                        <0x0 0x31101000 0x0 0x1000>;
                reg-names = "reg", "mem", "hpb";
                max-outbound-regions = <3>;
                max-functions = /bits/ 8 <1>;
        };

D2000 设备树（arch/arm64/boot/dts/phytium/pd2008-devboard-dsk.dts）：
        ep0: ep@0x29030000 {
                compatible = "phytium,pcie-ep-1.0";
                reg = <0x0 0x29030000 0x0 0x10000>,
                        <0x11 0x00000000 0x1 0x00000000>,
                        <0x0 0x29101000 0x0 0x1000>;
                reg-names = "reg", "mem", "hpb";
                max-outbound-regions = <3>;
                max-functions = /bits/ 8 <1>;
        };
```

此外还需关闭 SMMU 功能。

```
&smmu {
        status = "disabled";
};
```

RC 端：需要关闭 SMMU 功能。

```
&smmu {
        status = "disabled";
};
```

2.1.4 编译内核

```
make -j12
make modules_install I INSTALL_MOD_PATH=[module path]
```

使用编译生成的镜像、dtb 文件更新对应目标板镜像，将[module path]的内容全部复制到目标板/lib/modules 下面。

2.1.5 固件支持

EP 端的固件必须把 PCIe 控制器配置成 EP 模式，此为必要条件。无论是 Uboot 还是 UEFI 配置成 EP 模式，驱动均可支持，只是 UEFI 固件启动时需要使用设备树 DTB。

UEFI+DTB 启动参考如下链接：

https://gitee.com/phytium_embedded/phytium-linux-buildroot/wikis/UEFI%2BDTB%E5%90%AF%E5%8A%A8%E7%B3%BB%E7%BB%9F

固件代码的获取和固件制作，请咨询飞腾技术支持或 FAE。

2.1.6 编译 pcitest 工具

```
cd phytium-linux-kernel/tools/pci
make
```

在内核目录下编译 pcitest，将生成的 pcitest 二进制文件拷贝到 RC 端板卡/usr/bin 下，将 pcitest.sh 测试脚本拷贝到 RC 端板卡测试路径下。

2.1.7 硬件环境

使用 CARD_PCIe_GD_MCIO_X4_V0.1 子卡将两块 E2000Q DEMO 板连接起来，硬件环境如图 1-1：

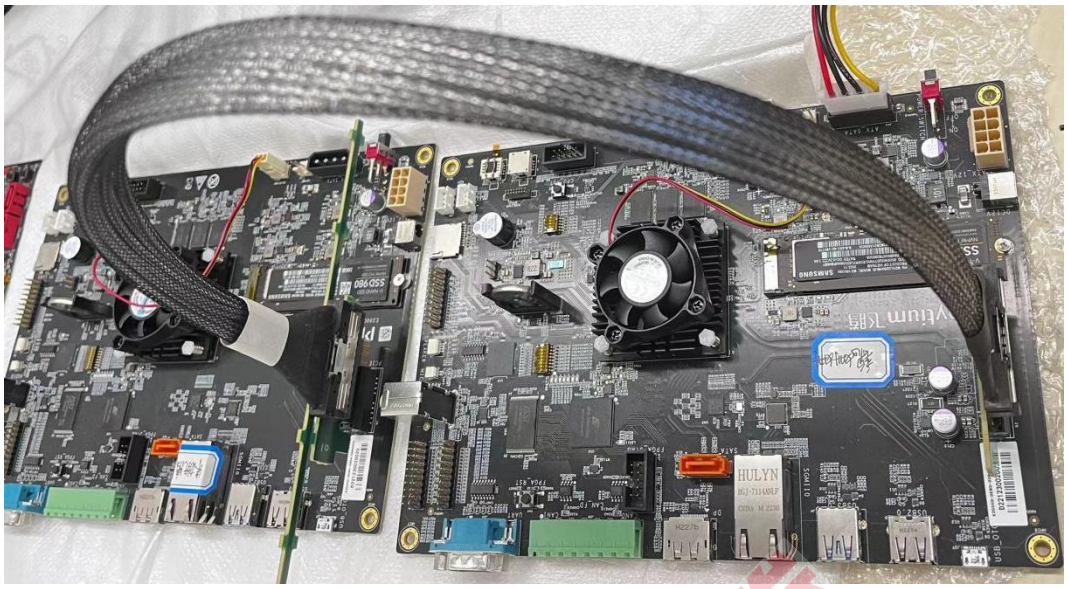

图 1-1 EP 功能测试硬件环境图

3 功能验证

3.1 EP 端

3.1.1 EP 端驱动加载

查询系统中的 EP 控制器设备列表:

```
E2000:
root@E2000-Ubuntu:~# ls /sys/class/pci_epc/
31040000.ep
D2000:
root@E2000-Ubuntu:~# ls /sys/class/pci_epc/
29030000.ep
```

下文中的 EP 控制器设备名称以 E2000 为例，D2000 下使用仅需把设备名称换成 29030000.ep。

查询 phytium-pcie-ep 驱动加载

```
root@E2000-Ubuntu:~/EP# ls -l /sys/bus/platform/devices/31040000.ep/
total 0
lrwxrwxrwx 1 root root      0 Nov 17 driver -> ../../../../bus/platform/drivers/phytium-pcie-ep
-rw-r--r-- 1 root root   4096 Nov 17 driver_override
-r--r--r-- 1 root root   4096 Nov 17 modalias
lrwxrwxrwx 1 root root   0 Nov 17 of_node
-> ../../../../firmware/devicetree/base/soc/ep@31040000
drwxr-xr-x 3 root root      0 Nov 17   pci_epc
drwxr-xr-x 2 root root      0 Nov 17   power
```

```
lrwxrwxrwx 1 root root      0 Nov 17   subsystem -> ../../../../bus/platform
-rw-r--r-- 1 root root 4096 Nov 17   uevent
```

查询系统中的 EP 功能驱动程序列表：

```
root@E2000-Ubuntu:/lib/modules/4.19.246-phytium-embeded-
v2.2+/kernel/drivers/pci/endpoint/functions# insmod pci-epf-test.ko
root@E2000-Ubuntu:/lib/modules/4.19.246-phytium-embeded-
v2.2+/kernel/drivers/pci/endpoint/functions#   ls /sys/bus/pci-epf/drivers
pci_epf_test
```

PCI -epf-test 驱动程序可用于测试 PCI 控制器的 EP 功能，目前支持的一些测试如表 2-1 所示:

表 2-1 EP 功能支持的测试

测试	描述
BAR	从 BAR 中写入和读取已知的模式
Interrupt (legacy/MSI/MSI-X)	从 EP 端触发中断(legacy/MSI/MSI- x)
Read	从 RC 端中的缓冲区中读取数据，并为该数据执行循环冗余检查(CRC)
Write	将数据写入 RC 端的缓冲区，并对该数据执行循环冗余检查
Copy	将数据从一个 RC 端的缓冲区复制到另一个 RC 缓冲区，并对该数据执行循环冗余检查

3.1.2 创建 pci-epf-test 设备

使用以下命令创建 pci-epf-test 功能：

```
mkdir /sys/kernel/config/pci_ep/functions/pci_epf_test/func1
echo 0x16c3 > /sys/kernel/config/pci_ep/functions/pci_epf_test/func1/vendorid
echo 0xedda > /sys/kernel/config/pci_ep/functions/pci_epf_test/func1/deviceid
echo 32 > /sys/kernel/config/pci_ep/functions/pci_epf_test/func1/msi_interrupts
```

为了使端点功能设备有用，它必须绑定到 PCI 端点控制器驱动程序。使用 configfs 将功能设备绑定到系统中存在的控制器驱动程序之一：

```
ln -s /sys/kernel/config/pci_ep/functions/pci_epf_test/func1
/sys/kernel/config/pci_ep/controllers/31040000.ep/
```

启动 EP 设备：

```
echo 1 > /sys/kernel/config/pci_ep/controllers/31040000.ep/start
```

创建成功之后可以在 pci-epf 总线下看到设备：

```
root@E2000-Ubuntu:~/EP# ls -l /sys/bus/pci-epf/devices/pci_epf_test.0/
total 0
lrwxrwxrwx 1 root root      0 Nov 17 08:19 driver -> ../../bus/pci-epf/drivers/pci_epf_test
drwxr-xr-x 2 root root      0 Nov 17 08:25 power
lrwxrwxrwx 1 root root      0 Nov 17 08:20 subsystem -> ../../bus/pci-epf
-rw-r--r-- 1 root root 4096 Nov 17 08:19 uevent
```

3.2 RC 端

3.2.1 RC 端驱动加载

如果 RC 端在 EP 端初始化之前就已经枚举了 pcie 设备，则需要重新枚举初始化后的设备，将之前的设备删除：

```
echo 1 > /sys/class/pci_bus/0000\:02/device/remove
```

重新枚举：

```
echo 1 > /sys/bus/pci/rescan
```

成功之后可以用 lspci -v 查看设备信息。

确认 ep 的驱动已经加载:

```
ls -l /sys/bus/pci/devices/0000\:02\:00.0/driver
/sys/bus/pci/devices/0000:02:00.0/driver -> ../../../../../../bus/pci/drivers/pci-endpoint-
test
```

3.2.2 执行测试程序

执行 pcitest.sh 脚本进行测试。

```
./pcitest.sh
```

检查输出 log，参考 log 如下：

```
root@E2000-Ubuntu:~/EP# ./pcitest.sh
BAR tests

BAR0:          OKAY
BAR1:          NOT OKAY
BAR2:          OKAY
BAR3:          NOT OKAY
BAR4:          OKAY
```

```
BAR5:                NOT OKAY          // E2000 和 D2000 不支持 bar1/3/5;

Interrupt tests

SET IRQ TYPE TO LEGACY:          NOT OKAY
LEGACY IRQ:       NOT OKAY          //E2000 和 D2000 不支持 legacy 中断
SET IRQ TYPE TO MSI:                OKAY
MSI1:               OKAY
MSI2:               OKAY
MSI3:               OKAY
MSI4:               OKAY
MSI5:               OKAY
MSI6:               OKAY
MSI7:               OKAY
MSI8:               OKAY
MSI9:               OKAY
MSI10:              OKAY
MSI11:              OKAY
MSI12:              OKAY
MSI13:              OKAY
MSI14:              OKAY
MSI15:              OKAY
MSI16:              OKAY
MSI17:              OKAY
MSI18:              OKAY
MSI19:              OKAY
MSI20:              OKAY
MSI21:              OKAY
MSI22:              OKAY
MSI23:              OKAY
MSI24:              OKAY
MSI25:              OKAY
MSI26:              OKAY
MSI27:              OKAY
MSI28:              OKAY
MSI29:              OKAY
MSI30:              OKAY
MSI31:              OKAY
MSI32:              OKAY

Read Tests
```

Phytium飞腾

```
SET IRQ TYPE TO MSI:                OKAY
READ (        1 bytes):             OKAY
READ (     1024 bytes):             OKAY
READ (     1025 bytes):             OKAY
READ (1024000 bytes):               OKAY
READ (1024001 bytes):               OKAY

Write Tests

WRITE (        1 bytes):            OKAY
WRITE (     1024 bytes):            OKAY
WRITE (     1025 bytes):            OKAY
WRITE (1024000 bytes):              OKAY
WRITE (1024001 bytes):              OKAY

Copy Tests

COPY (        1 bytes):             OKAY
COPY (     1024 bytes):             OKAY
COPY (     1025 bytes):             OKAY
COPY (1024000 bytes):               OKAY
COPY (1024001 bytes):               OKAY
```

4 注意事项

1. EP 端和 RC 端使用的内核版本必须一致，保证两端 pci-endpoint-test 驱动接口匹配。
2. pcitest 工具必须在对应版本的内核源码中编译。

以上仅为 EP 驱动功能验证需要，客户实际应用场景中没有这些限制。

5 常见问题

1. 在内核代码中打上 EP 的 patch 后设备冲突，驱动加载失败

gitee 的内核源码中已经包含了 EP 功能的补丁，在 drivers/pci/controller/pcie-phytium-ep.c 路径下，不需要再打上额外的 patch 了。

A 术语

pcie 终端设备　EP （Endpoint device）

pcie 根节点　RC （PCI Express root complex）

循环冗余检查　CRC （Cyclic redundancy check）

B 更新记录

发布日期	说明
2023-11-17	v1.0 初稿
2025-3-20	v1.1 增加 D2000 相关内容

Phytium飞腾